SITE AUTOMATION

How It Fits Together

Inventory, variables, the YAML files and surveys - and how to update the repository safely

Site Automation teaching guide | site-automation 0.78.1 | October 2026

What this is. How the pieces of site automation fit together: what the code, the inventory, the credentials and the survey each do; how one setting travels from its default in a YAML file to a host; how groups work (and a host in several of them); what a survey is and where to find it; and how to update the repository at work without overwriting the wrong things.

Read this first

This guide explains the ideas. The *Operations Runbooks Setup Guide* has the clicks, one step at a time. Every setting you can change, with its default, is in the *Settings Reference* PDF.

Rendered from `docs/HOW_IT_FITS_TOGETHER.md` and `playbooks/group_vars/` in the site-automation repository (version 0.78.1).

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

1. How it all fits together 3

The four kinds of pieces . . . 3
Reading YAML in two minutes . . . 4
The YAML files in this repository, and which ones you touch . . . 5
Follow one setting from its default to a host . . . 5
The inventory: one inventory, many groups, and a host in several groups . . . 7
Is it modular? Yes, in three ways . . . 8
Surveys: the questions at launch (run-time variables) . . . 9
Updating the repository at work, without overwriting the wrong things . . . 10
Cheat sheet: where does it go? . . . 13

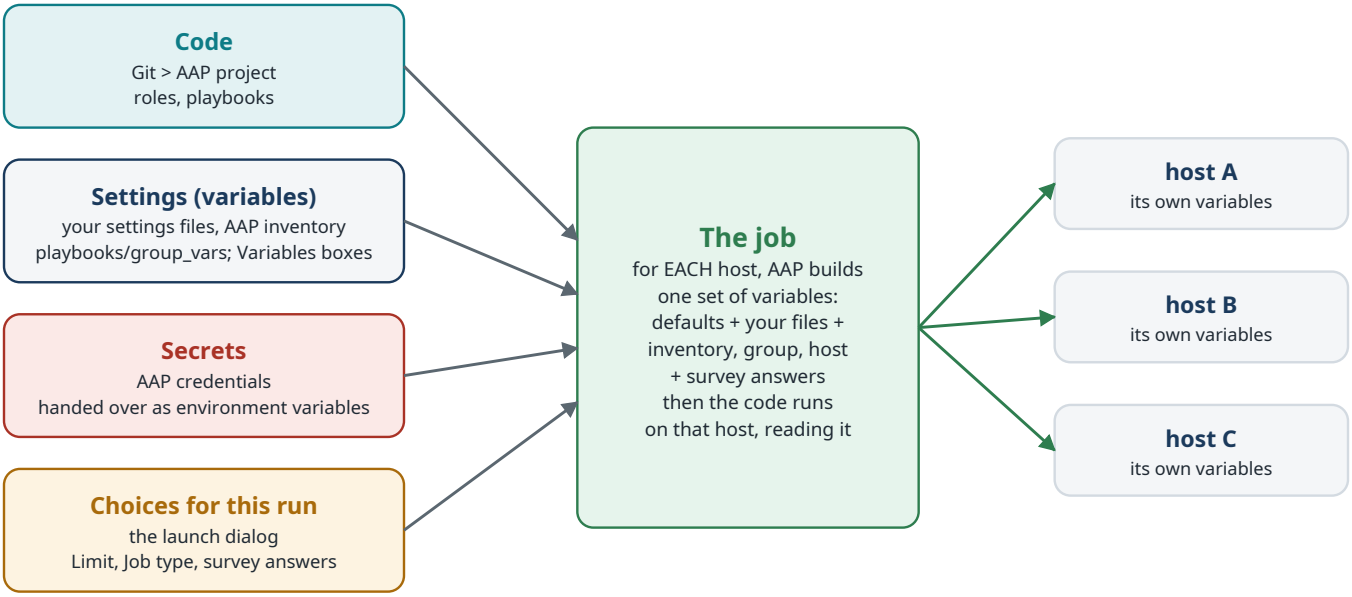
Appendix: your settings files (playbooks/group_vars/) 14

all.yml (every host) . . . 14
aap.yml (AAP group aap) . . . 17
idm.yml (AAP group idm) . . . 17
logstash.yml (AAP group logstash) . . . 17
mariadb.yml (AAP group mariadb) . . . 17
mysql.yml (AAP group mysql) . . . 18
netapp.yml (AAP group netapp) . . . 18
rhel8_all.yml (AAP group rhel8_all) . . . 18
sat.yml (AAP group sat) . . . 19
stigman.yml (AAP group stigman) . . . 19

1. How it all fits together

This repository has many files, and AAP has many screens. Every one of them does one of **four jobs**. Once you know which job a piece does, you know where to look, and what you may change.

The four kinds of pieces



Four kinds of pieces, each in its own place. AAP combines them into one set of variables per host when a job starts.

Piece	What it holds	Where it lives	Who changes it	Example
Code	*what to do*, step by step	Git (this repository), seen by AAP as the **project**	nobody, except to install a new release	`roles/check_disk/tasks/main.yml`
Settings (variables)	*the numbers and names that are different at your site*, or for some hosts	your settings files in `playbooks/group_vars/` (one per AAP group), or the AAP **inventory**'s **Variables** boxes	you	`check_disk_warn_pct: 80`
Secrets	passwords, keys, tokens	AAP **credentials**	you, once	the ServiceNow API password
Choices for this run	which hosts, which checks, dry run or real	the **launch** dialog: Limit, **survey**	whoever clicks Launch	`health_checks: weekly`

When a job starts, AAP puts the four together **for each host**:

1. the code, from the project;

2. the settings, from your settings files and the inventory. Each host gets the values of its own groups and its own Variables box;
3. the secrets, from the credentials, handed over as environment variables;
4. your answers, from the survey.

The code then reads from that host's combined set of variables. That is the whole machine.

The one rule that keeps upgrades painless: never edit a file that came with a release, except **your** files: your settings in `playbooks/group_vars/` and your POA&M list `poam/poam.csv`. Everything else you want different goes in AAP: secrets in credentials, choices in surveys. A new release then replaces our files, and your settings are untouched.

Reading YAML in two minutes

Every settings box in AAP, and most files here, are YAML: names and values, one per line.

YAML

```
check_disk_warn_pct: 85                # a setting: name, colon, space, value
check_services_required:               # a list: one "- item" per line...
  - sshd
  - crond
check_services_required: [sshd, crond] # ...or the same list on one line
check_network_tcp:                     # a list of small tables (name / host / port each)
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_disk_overrides:                  # a table inside a table
  /var/log/audit: {warn: 70, crit: 85}
```

- **Indentation is spaces, never tabs.** It matters: the things that belong to a name are indented further than the name.
- `#` starts a comment. Everything after it on that line is ignored.
- `{{ something }}` inside a value means "fill in the variable `something` here". You see it in the code; you rarely need it in settings.
- AAP will not save a Variables box whose YAML it cannot read, so a typo shows up right away.

The YAML files in this repository, and which ones you touch

Path	What it is	Edit it?
`playbooks/group_vars/*.yml`	**your settings**, one file per AAP group (`mariadb.yml` = group `mariadb`, `all.yml` = every host). Placeholders start with `#`	**yes**: this is where you change settings
`playbooks/*.yml`	the files AAP runs, one per job template. Each says *which hosts* and *which roles*	**no**
`roles/<role>/defaults/main.yml`	**every setting of that role**, with its default value and a comment saying what it does. The menu of what you can change (the same list, ready to read: VARIABLES_REFERENCE.md)	**no**: read it, then set values in your settings files or in AAP
`roles/<role>/tasks/*.yml`	the steps. They read the settings	**no**
`roles/<role>/meta/main.yml`	the role's name tag	**no**
`aap/credential_types/*.yml`	forms you paste into AAP (Credential Types)	**no**
`inventories/example/`	a sample inventory. Copy blocks from it; AAP does not read it	**no**
`ansible.cfg`	tells Ansible where the roles and filters are. AAP uses it too, but ignores its `inventory =` line	**no**
`vendor/act/`	ACT, copied in from its own release	**no**
`docs/`, `docs/pdf/`	these guides	**no**
`tests/`, `.github/`, `.ansible-lint`, `.yamllint`	automatic checks of the code	**no**
`poam/poam.csv`	**your** POA&M list	**yes**

Two places can hold the same setting: your settings file in `playbooks/group_vars/`, and AAP's Variables box. **Keep each setting in one place.** If both have it, the file wins over an AAP *group* or *inventory* box (a *host's* box in AAP beats a group file; a template's extra variables and survey answers beat everything). The full order is in VARIABLES.md. The file is also the easier place: you edit it in VS Code, and Git remembers every change.

Every setting there is, with its default and what it does, is in one place: VARIABLES_REFERENCE.md - in print, the *Settings Reference* PDF.

Follow one setting from its default to a host

Take `check_disk_warn_pct`: warn when a filesystem is this many percent full.

1. **The default.** `roles/check_disk/defaults/main.yml` says `check_disk_warn_pct: 85`. With nothing else set, every host warns at 85%.
2. **The code reads it.** `roles/check_disk/tasks/main.yml` compares each filesystem with `check_disk_warn_pct`. You never change this part.
3. **Change it for every host.** Add `check_disk_warn_pct: 80` to `playbooks/group_vars/all.yml` (or to **Inventories › your inventory › Edit inventory › Variables**). Now every host warns at 80%.
4. **Change it for a group.** Add `check_disk_warn_pct: 90` to the group's file, for example `playbooks/group_vars/mariadb.yml` (or the group's **Edit group › Variables** in AAP). Hosts in that group warn at 90%; all others still at 80%.
5. **Change it for one host.** Open the host › **Edit host › Variables**: `check_disk_warn_pct: 95`. Only that host warns at 95%.
6. **For one run.** A survey answer, or the template's own Variables box, beats all of the above, for that run only. This setting has no survey question, so it simply comes from your settings files or the inventory.



Where a value can be set, weakest at the bottom; the highest place that sets it wins. A host with its own value gets 95, the other hosts in that group 90, everyone else 80 - unless one of your settings files sets it too: those beat the AAP inventory and group boxes. The top two rows use health_checks: it is chosen per run, not per host. Full order: docs/VARIABLES.md.

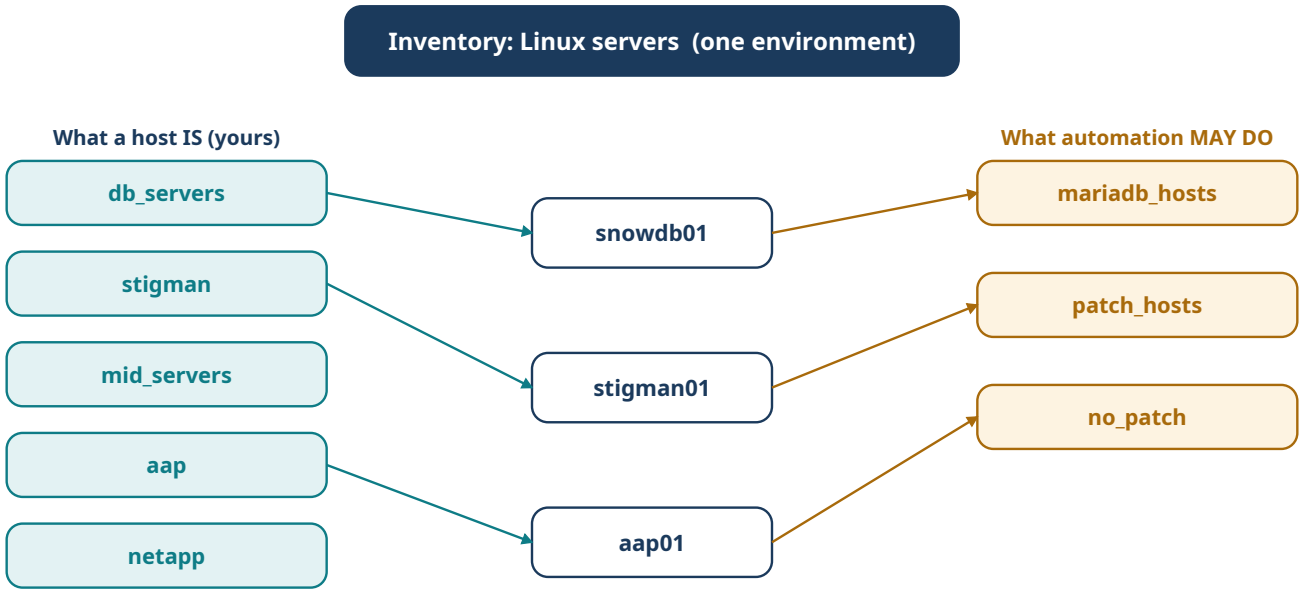
The rule in one line: **the closer to the host, the stronger, except that this project's settings files (`playbooks/group_vars/`) beat AAP's group and inventory boxes. An answer given at launch beats everything, for that run.** The full order: VARIABLES.md.

Two things that trip people up:

- **The same setting in two groups of one host.** A host is in both `db_servers` and `stigman`, and both groups set `check_disk_warn_pct`. Ansible then takes the group whose name sorts last alphabetically, which is easy to miss. Set such a value on one group only, or on the host.
- **Some variables are per run, not per host.** Set them at launch (survey) or on the template. For `target` and `health_checks`, an inventory value is even ignored:

Variable	Set it in	Why
`target` (which hosts a playbook runs on; empty = every host, and `stigman` for service watch)	the template's **Variables** box	it picks the hosts *before* any inventory setting is read
`health_checks`	the Health check **survey**	the playbook's own value beats the inventory
`ts_area`, `ts_service`, `ts_target`, `ts_since`	the Troubleshoot **survey**	chosen per problem
`use_act`, `site_act_level`	a **survey** (or the inventory, as a fixed default)	chosen per run
everything named `check_*`, `patch_*`, `site_act_*`	your settings files (`playbooks/group_vars/`), or a group's or host's Variables	facts about your site and hosts. The ACT provider and models: defaults in the role; to change, un-comment the lines in `all.yml`
`servicenow_*`, `poam_*`	`playbooks/group_vars/all.yml` (the placeholders are there), or the **inventory's own Variables** box, not both (the file wins). Not a group's box	those jobs run on AAP itself, which is in no group, so it only gets `all.yml` and inventory-level settings

The inventory: one inventory, many groups, and a host in several groups



Each host is in several groups at once, and gets the Variables of every one of them.

One inventory, many groups. snowdb01 is in db_servers (yours) and also in mariadb_hosts, so the MariaDB check runs there. Every host is also in the built-in group all: the checks run there when a template has no target.

- **One inventory holds all the hosts of one environment**, for example `Linux servers`. Make a second inventory only for a separate environment (test and production). Never make one to sort hosts by role: that is what groups are for.
- **Groups are labels.** A host can be in as many groups as you like. Adding a host to a group never removes it from its other groups.

- **Groups can contain groups.** Put your `db_servers` group inside `mariadb_hosts` and every database host is in `mariadb_hosts` too, now and whenever you add one later.
- **Settings follow the groups.** A host gets the Variables of every group it is in (and of their parent groups), then its own.

Three kinds of groups work well together, and you already have the first kind:

Kind	Examples	Used for
What a host **is**	your `db_servers`, `stigman`, `mid_servers`, `aap`, `netapp`	settings that belong to that kind of host (the MariaDB container name, the MID Server services' names)
What automation **may do** to it	`patch_hosts`, `no_patch`, `aap_hosts`, `netapp_console_hosts`	safety: what gets patched, what ACT may only diagnose
Everything	`all` (built in: you never create it)	where the checks run when a template has no `target`

Should you make more groups? Yes: a few more **groups**, in your **one** inventory. You have two ways to connect your groups to the names the runbooks use. Both work, so pick one:

- **Way A: add the runbook names as groups, with your groups inside.** A few clicks, and the guides match what you see. For example, create `mariadb_hosts` and put `db_servers` inside it.
- **Way B: tell the runbooks your names.** No new groups. In your settings file `playbooks/group_vars/all.yml`, or in the inventory **Variables** box in AAP (one of them, not both): a list you write replaces the shipped one, so repeat the names you keep; `aap` and `aap_hosts` are always protected. (Updated from 0.4.1? Your `all.yml` still sets these lists; comment them out there before you use the AAP box, or the file wins.)

YAML

```yaml
site_act_diagnose_only_groups: [aap, netapp]   # ACT never fixes these
patch_never_reboot_groups: [aap]               # patching never reboots these
patch_never_patch_groups: [aap, netapp]        # patching skips these
```

Then put `target: <the group you patch>` in the Variables box of Patch hosts. (The checks run on every host unless a template sets `target`, for example `target: linux_servers`.) On your database group set `check_mariadb_enabled: true`.

Is it modular? Yes, in three ways

1. **Checks are separate pieces.** Each check is its own role (folder). You pick them by name per run (survey), per schedule, or per workflow step. A check of your own is one new folder (ADDING_ACT.md, "Write your own check").
2. **Settings are separate from code.** Every setting has a default in its role, and you override it at the level that fits: the inventory for everyone, a group for some, a host for one. **It is the same variable name at every level.** So set a value once at the top, and override it only where a group or host is different.
3. **Results are separate from what reads them.** Every check reports findings the same way. The report, ServiceNow tickets and ACT therefore work with every check, including yours.

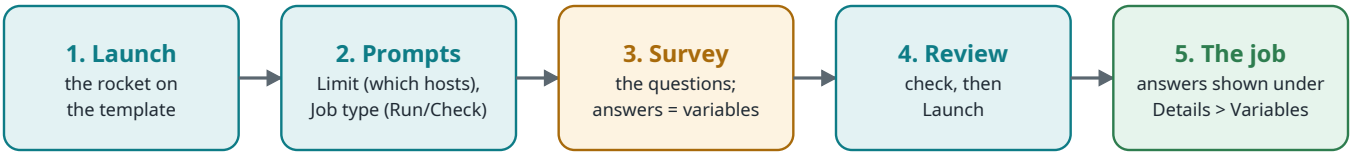
Surveys: the questions at launch (run-time variables)

What it is. The questions a job template asks when someone clicks **Launch**. Each answer becomes a variable for **that run only**. So yes: a survey is how you set variables at run time. It is the safe way: you decide the question, the allowed answers and the default, and people cannot type anything else.

Where it is. Automation Execution › Templates › open the template **› Survey** tab.

- **Create survey question** adds one. **Question** is what people read. **Answer variable name** is the variable it sets (for example `health_checks`). **Answer type** is, for example, *Text*, *Integer*, *Multiple Choice (single select)*, *Multiple Choice (multiple select)*. Then the choices, a **Default answer**, and **Required**.
- **The switch at the top of the tab** turns the survey on. A survey that is off is never asked.

At launch:



Schedules and workflow steps skip 2-4: their answers are set once (Prompts / Survey step).

What happens when you click Launch. The survey is where you set variables for one run.

1. Click the rocket (**Launch**).
2. AAP first asks anything the template "prompts on launch" for, such as the **Limit** (which hosts) or the **Job type** (Run, or Check for a dry run).
3. Then the **survey** page: answer, or keep the defaults.
4. **Next › Launch.** The job's **Details** tab shows the answers under **Variables**. So you can always see what a run was asked to do.

In schedules and workflows there is nobody to answer. So you give the answers once: in the schedule's **Prompts** step, or in the workflow step's **Survey** step. Without them, the **defaults** are used.

Survey, template Variables, or inventory Variables?

	Survey	Template Variables box	Inventory / group / host Variables
Set by	whoever launches	whoever owns the template	whoever owns the inventory
Applies to	this run	every run of this template	every run, on those hosts
Example	`health_checks: weekly`	`target: linux_servers`	`check_disk_warn_pct: 80`

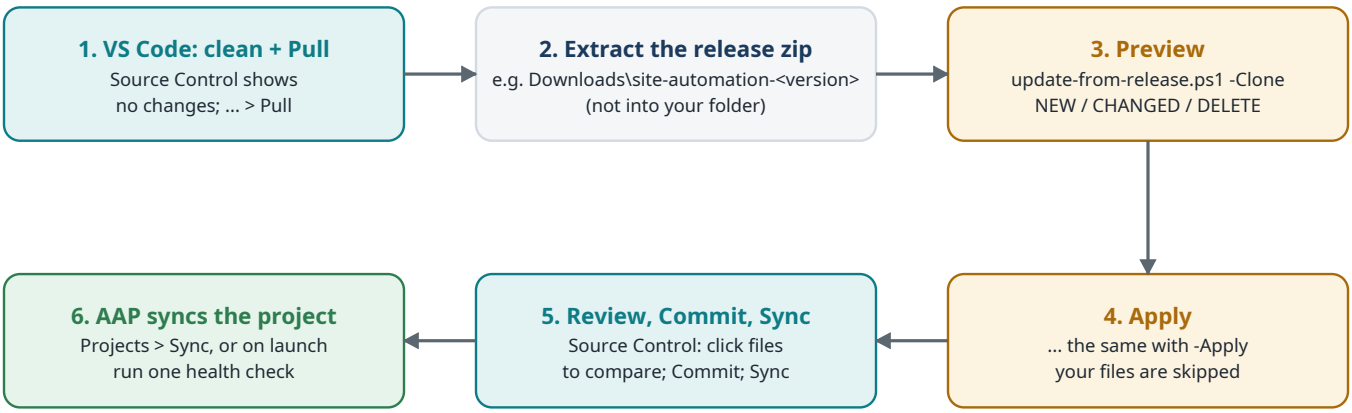
Never put a password in a survey or a Variables box. Secrets go in **credentials**.

The survey questions these runbooks use:

Template	Variable	Answers
Health check	`health_checks` (multiple select)	`daily`, `weekly`, `all`, or check names

Template	Variable	Answers
Troubleshoot	`ts_area`, `ts_service`, `ts_target`, `ts_since`	area; unit name; host:port; how far back
Certificate report	`check_certs_warn_days`	days (default 30)
POA&M status	`poam_stigman_api`	STIG Manager API URL, or blank
Patch hosts, Windows patch	`patch_security_only` (Linux), `automatic_restarts`	`false`/`true`; `no`/`yes` (default `no`: no automatic restarts)
(with ACT) Health check, Troubleshoot	`use_act`, `site_act_level`, `site_act_provider`, `site_act_model`	`no`/`yes`; `explain`/`diagnose`/`self-heal`; `genai`/`asksage`/`genai-beta`; a model id

Updating the repository at work, without overwriting the wrong things



Yours, never touched: playbooks\group_vars + host_vars, poam.csv, inventories\site, .site-local paths.

Updating the work repository from Windows. Nothing reaches your Git server (or AAP) until you click Sync Changes, after you have seen every change.

Think of your work repository as **our files + your few files**. A new release replaces our files and never touches yours. Your files are:

- `playbooks/group_vars/` and `playbooks/host_vars/` (your settings files). A new release may add a settings file you do not have yet; it never changes one you have;
- `poam/poam.csv` (your POA&M list);
- `inventories/site/`, only if you ever keep an inventory in Git (you keep yours in AAP);
- anything you added yourself, such as your own check role. List those paths in a file called `.site-local` at the top of your repository folder, one per line, for example `roles/check_tmp/`.

These steps are for **Windows with VS Code**: your repository folder is open in VS Code, and VS Code does the Git part (commit, push). Nothing here needs Git on the RHEL servers. AAP pulls from your Git server by itself.

Method 1: the update script (recommended)

1. **Get your folder up to date.** In VS Code, open your site-automation folder (**File › Open Folder**). Open **Source Control** (Ctrl+Shift+G): it must show **no changes**. Commit or discard anything listed there first. Then **… › Pull**, to get whatever is newest on the server.
2. **Extract the new release somewhere else.** Right-click `site-automation-<version>.zip` › **Extract All** › for example `Downloads\site-automation-<version>` (`<version>` is the number in the file name, for example `0.6.0`). **Not** into your repository folder.
3. **Preview.** It changes nothing. **Terminal › New Terminal** (it is PowerShell), then:

```
$rel = "$HOME\Downloads\site-automation-<version>"          # where you extracted the release
& "$rel\scripts\update-from-release.ps1" -Clone "C:\git\site-automation"
```

`-Clone` is your repository folder: the one VS Code has open. The script lists every file as `NEW`, `CHANGED`, `DELETE`, or `YOURS` (a settings file you do not have yet: added once, then yours). **Read the DELETE lines.** If one is yours, add its path to `.site-local`, commit that, and preview again.

4. **Apply.** The same command, with `-Apply` at the end:

```
& "$rel\scripts\update-from-release.ps1" -Clone "C:\git\site-automation" -Apply
```

5. **Review in VS Code. Source Control** now lists every file the update changed. Click one to see the old and new versions side by side. See something of yours? Right-click it › **Discard Changes** keeps your version.
6. **Commit and push.** Type a message (`site-automation <version>`), click **Commit**, and answer *Yes* if VS Code asks to stage all changes. Then click **Sync Changes**. Only now does the new version reach your Git server.
7. **In AAP: Projects › site-automation › Sync**. With **Update revision on launch** ticked, the next job syncs by itself. The project page shows the new revision (the commit id).
8. **Check:** run **Health check** on one host.

Changed your mind before step 6? Source Control › … › Changes › Discard All Changes. Nothing left your PC.

"Running scripts is disabled" or **"not digitally signed"** when you run the script? Run it through a new PowerShell that is allowed to run it this one time (add `-Apply` for step 4):

```
cd $rel
powershell -ExecutionPolicy Bypass -File .\scripts\update-from-release.ps1 -Clone
  "C:\git\site-automation"
```

If your organization blocks that too, use Method 2.

Method 2: copy and paste the release (no scripts)

Most people do this: copy the release over the repository. It is safe as long as **your own files are not in what you paste**. The release has example copies of them, and pasting those would replace yours: your mail relay, your exclusion lists, every setting you made.

1. **Your folder is up to date and clean.** VS Code, **Source Control** (Ctrl+Shift+G): no changes listed. Commit or discard anything there first. Then **… › Pull**.
2. **Extract the release somewhere else**: right-click `site-automation-<version>.zip` › **Extract All** › e.g. `Downloads\site-automation-<version>`. Not into your repository folder.

3. **In the extracted folder, delete what is yours** (only the release's examples are deleted):
   - the folder `playbooks\group_vars` (your settings: `all.yml`, `aap.yml`, ...);
   - the folder `playbooks\host_vars`, if there is one;
   - `poam\poam.csv`;
   - `inventories\site`, if you keep an inventory in Git;
   - `.site-local`, and every path listed in yours.
4. **Copy and paste.** In the extracted folder: Ctrl+A, Ctrl+C. In your repository folder: Ctrl+V › **Replace the files in the destination**.
5. **The safety net: look before you commit. Source Control** now lists what the release changed (**M**) and added (**U**, new files).
   - One of your files in the list, e.g. `playbooks/group_vars/all.yml` (step 3 forgotten)? Right-click it › **Discard Changes**. Your version is back.
   - The **U** files are new parts of the release: keep them.
6. **Commit and push.** Message `site-automation <version>` › **Commit** (answer **Yes** when VS Code asks to stage all changes: that includes the new files) › **Sync Changes**.
7. **In AAP: Projects › site-automation › Sync**. Then run one job to check: the health check on one host, or a job the release changed.

The same steps work for a zip of only the changed files.

Copying never deletes: a file the release *removed* stays in your folder. The `CHANGELOG.md` says when a release removes one; delete it by hand.

New settings in a release have their defaults in the roles, so they work without you doing anything. To change one, add it to **your** `playbooks/group_vars/all.yml` (or a template's Variables); VARIABLES_REFERENCE.md lists them all.

Line endings are taken care of. The repository's `.gitattributes` keeps Linux line endings (LF) in Git, even when you edit and commit on Windows. That matters: the playbooks run shell commands on RHEL, and those break with Windows line endings. VS Code shows `LF` in its status bar; leave it that way.

On Linux instead of Windows? `scripts/update-from-release.sh` does the same as Method 1: `/tmp/site-automation-<version>/scripts/update-from-release.sh ~/site-automation`, then again with `--apply`, then `git status`, `git commit`, `git push`.

First time, with no repository at work yet? Create an empty repository on your Git server, clone it in VS Code (**Source Control › Clone Repository**), copy the extracted release into that folder, then **Commit** and **Publish/Sync**. SETUP_AAP.md step 1 covers the inventory part.

Cheat sheet: where does it go?

You want to...	Put it in
add a server	AAP inventory › **Hosts** (and into its groups)
say what a server is (database, STIG Manager, MID...)	AAP inventory › **Groups**
change a threshold or name for every host	`playbooks/group_vars/all.yml` (or inventory › **Edit inventory › Variables**)
...for some hosts	the group's file, `playbooks/group_vars/<group>.yml` (or the group › **Edit group › Variables**)
...for one host	the host › **Edit host › Variables**
store a password or key	**Credentials**
let people choose at launch	the template's **Survey**
make a template always run on your group	the template's **Variables** box: `target: <group>`
pick which checks a schedule runs	the schedule's **Prompts** (survey answers)
update the code	a new release, with `scripts/update-from-release.ps1` in VS Code (above)
change what a check does	a new check of your own (ADDING_ACT.md); never edit ours

Appendix: your settings files (playbooks/group_vars/)

One file per AAP group; `all.yml` is every host. Lines starting with `#` are off: delete the `#` and replace `CHANGE-ME` to turn a placeholder on. Edit them in VS Code, Commit, Sync Changes - AAP uses them on the next run. These are YOUR files: updates never change them.

all.yml (every host)

YAML

```
# YOUR SETTINGS FOR EVERY HOST - and for the jobs that run on AAP itself (ServiceNow tickets,
# ServiceNow health, POA&M status).
#
# To change a setting: edit this file in VS Code, Commit, Sync Changes. AAP uses it on the next
# run (the project syncs itself when "Update revision on launch" is ticked).
# A line starting with "#" is OFF. To turn a placeholder on: delete the "# " in front of it (and
  in
# front of the lines that belong to it) and replace CHANGE-ME with your value. Keep the
# indentation exactly as shown: spaces, never tabs.
#
# The files in this folder are YOURS: the update script adds them the first time and never
# changes them again. File name = AAP group name (mariadb.yml -> the hosts in group "mariadb").
# What each setting does: roles/<role>/defaults/main.yml. Keep each setting in ONE place - here,
# or in AAP's Variables boxes (docs/USING_YOUR_AAP_INVENTORY.md) - not both: a value in this file
# silently WINS over the same setting in the AAP inventory's Variables box (the jobs print a
  notice).

# ---- Safety: hosts automation must be careful with (your AAP group names) ------------------
# ALREADY ON, no need to copy them here: the built-in lists in roles/patch/defaults/main.yml and
# roles/site_act/defaults/main.yml cover aap, aap_hosts, sat, idm, netapp and
  netapp_console_hosts.
# AAP itself (aap, aap_hosts) can NOT be removed from the protection. Un-comment a line only to
# use YOUR OWN group names (the list you write REPLACES the built-in one, so repeat the names you
  keep):
# site_act_diagnose_only_groups: [aap, aap_hosts, sat, idm, netapp, netapp_console_hosts,
  my_group]
# patch_never_patch_groups: [no_patch, aap, aap_hosts, sat, netapp, netapp_console_hosts,
  my_group]
# patch_never_reboot_groups: [aap, aap_hosts, sat, netapp, netapp_console_hosts, my_group]
# Linux and Windows patching both honour the two patch lists.

# ---- Patching (Linux and Windows): NO automatic restarts unless this is true
  --------------------
# automatic_restarts: false
```

YAML (continued)

```
# ---- ACT: which provider, and the model for each ------------------------------------------------
# Defaults (roles/site_act/defaults/main.yml): provider genai; models genai
  gemini-4-argon-withtools,
# asksage gpt-5.6-sol-gov. Un-comment only to change them.
# site_act_provider: genai               # genai | genai-beta (asksage: Ask Sage is retired for
  now)
# site_act_models:                       # the model follows whichever provider is picked
#   genai: gemini-4-argon-withtools
#   asksage: gpt-5.6-sol-gov
# Ask Sage URL (empty = ACT's default,
  https://api.genai.army.mil/server/openai/v1/chat/completions):
# site_act_urls:
#   asksage: https://CHANGE-ME/server/openai/v1/chat/completions
# Self-heal only (site_act_level: self-heal): the fixes ACT may run by itself, each a regular
# expression that must match the WHOLE command (default: none - every fix waits for approval):
# site_act_allow: ['systemctl restart (chronyd|rsyslog|crond)']

# ---- What every host must be able to reach ------------------------------------------------------
# check_network_tcp:
#   - {name: IdM LDAPS, host: CHANGE-ME.yoursite.mil, port: 636}
# check_logging_remote:
#   - {name: logstash, host: CHANGE-ME.yoursite.mil, port: 514}

# ---- Audit volume (the weekly auditd check)
  ------------------------------------------------------
# The account your vulnerability scanner logs in with makes most of the audit records during a
# scan: list it here so it does not count toward the audit volume warning (the report still says
# how much it made). Names or UIDs, the same on every host; a host without the account skips it.
# check_auditd_exclude_accounts: [CHANGE-ME-scanner-account]
# check_auditd_volume_warn_mb_day: 200            # MB of audit records a day, excluded accounts
  not counted

# ---- VMware jobs (playbooks/vm_*.yml, docs/VMWARE.md) -------------------------------------------
```

YAML (continued)

```
# Restart, shut down and VLAN change never touch the AAP server's VM (found through the groups aap
  /
# aap_hosts). Name it here too when its VM name differs from its host name - and vCenter's own VM:
# vmware_protected_vms: [CHANGE-ME-aap-vm-name, CHANGE-ME-vcenter-vm-name]
# vmware_validate_certs: true                    # false only until the execution environment
  trusts vCenter's CA

# ---- Emailed reports (docs/EMAIL_REPORTS.md) ------------------------------------------------
# The mail relay for every job that emails a report. Who gets a report: report_email_to in each
# TEMPLATE's Variables (here, every report job would email those people every time it runs).
# report_email_smtp_host: CHANGE-ME.yoursite.mil   # your mail relay
# report_email_from: CHANGE-ME@yoursite.mil        # the sender
# report_email_security: starttls                  # starttls (25/587) | ssl (465, with
  report_email_smtp_port: 465) | none
# The look of every report email: standard | command | portal | glass (your logo, the job's last
  runs from AAP - "The look")
# report_email_theme: command                      # or portal, glass
# The look of the report website (docs/WEB_REPORTS.md): atlas (the default: light and dark, made
  for a browser - never
# an email), or an email look: standard | command | portal | glass
# report_web_theme: atlas
# report_logo_file: files/report_logo.png          # = playbooks/files/report_logo.png, in your
  repository
# Every report also as a web page (docs/WEB_REPORTS.md) - with or without the email:
# report_web_dest: https://CHANGE-ME.yoursite.mil/upload   # an upload address (PUT), or a mounted
  folder: /mnt/reports
# report_web_url: https://CHANGE-ME.yoursite.mil           # the address people open
# Or on a Windows server, over WinRM (the login: a "Report website" credential on the report
  templates):
# report_web_host: CHANGE-ME.yoursite.mil                 # the server; report_web_dest is then
  its folder:
# report_web_dest: 'D:\reports'

# ---- The account AAP logs in as -------------------------------------------------------------
```

YAML (continued)

```
# check_accounts_nopasswd_allowed: [CHANGE-ME-service-account]
# check_accounts_expiry_watch: [CHANGE-ME-service-account]

# ---- ServiceNow tickets (set up and test: docs/SERVICENOW_SETUP.md) -----------------------------
# servicenow_assignment_group: CHANGE-ME           # a group that exists in ServiceNow
# servicenow_min_severity: warning                 # or critical: tickets for critical findings
  only
# servicenow_caller: CHANGE-ME                     # optional; empty = the API account is the
  caller
# servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"   # if the EE does not trust
  the instance's CA

# ---- POA&M: STIG Manager cross-check (optional)
  ------------------------------------------------
# poam_stigman_api: https://CHANGE-ME/api
```

aap.yml (AAP group aap)

YAML

```yaml
# YOUR SETTINGS FOR THE AAP HOSTS (AAP group aap). ACT never fixes anything here, and patching
# skips these hosts (all.yml).
check_disk_overrides:
  /var: {warn: 80, crit: 90}            # AAP keeps job output and container images under /var
check_certs_endpoints:
  - {name: AAP web UI, host: 127.0.0.1, port: 443}
```

idm.yml (AAP group idm)

YAML

```yaml
# YOUR SETTINGS FOR THE IDM SERVERS (AAP group idm). ACT never fixes anything here (all.yml).
check_services_required_extra: [ipa]     # the IdM services, as one unit (systemctl status ipa)
check_certs_endpoints:                   # IdM's certificates expiring breaks every login
  - {name: IdM web UI, host: 127.0.0.1, port: 443}
  - {name: IdM LDAPS, host: 127.0.0.1, port: 636}
```

logstash.yml (AAP group logstash)

YAML

```yaml
# YOUR SETTINGS FOR THE LOGSTASH HOSTS (AAP group logstash).
check_services_required_extra: [logstash]
```

mariadb.yml (AAP group mariadb)

YAML

```yaml
# YOUR SETTINGS FOR THE MARIADB HOSTS (AAP group "mariadb"; the example inventory calls it
# "mariadb_hosts" - the database check accepts both names, mysql / mysql_hosts / database_hosts
  too,
# and any host you give a container name; MySQL hosts: playbooks/group_vars/mysql.yml).
# A settings file applies to the group with the same name as the file: if your MariaDB hosts are
# in a group with another name, copy this file to playbooks/group_vars/<that group>.yml.
# Full guide (rootless containers, several containers, the credential): docs/MARIADB.md.
#
# To turn the MariaDB check on for MariaDB in podman containers:
#   1. On a MariaDB host, run: sudo podman ps -a   - and note the container NAME(S).
#      (Rootless containers belong to a user: sudo -u THAT_USER podman ps -a)
#   2. Below, delete the "# " in front of the lines you need and replace CHANGE-ME.
#   3. In AAP, attach the "MariaDB monitor" credential (the read-only DB account) to the Health
#      check, Database health and Troubleshoot job templates. Credentials attach to templates, not
  to groups; the
#      check only uses it on the hosts that have a database.
# check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME                 # one container ...
# check_mariadb_containers: [CHANGE-ME, CHANGE-ME-2]  # ... or several on the same host
# check_mariadb_container_discover: true             # ... or find every mariadb/mysql/percona
  container
#   (with none of them: no database server on the host, or a mariadb.service that runs podman,
#    and the container is found by itself - 0.7.0 and later)
# check_mariadb_container_user: CHANGE-ME            # rootless: the account that owns the
  containers
```

mysql.yml (AAP group mysql)

YAML

```
# YOUR SETTINGS FOR THE MYSQL HOSTS (AAP group "mysql"; the example inventory calls it
  "mysql_hosts" -
# the database check accepts mariadb_hosts, mysql_hosts, database_hosts, mariadb and mysql, and
  any host
# you give a container name). A settings file applies to the group with the same name as the file:
  if your
# MySQL hosts are in a group with another name, copy this file to playbooks/group_vars/<that
  group>.yml.
# The check is the same one as for MariaDB (MariaDB / MySQL 5.7 / 8.0 / 8.4 are told apart by
  themselves).
# Full guide (containers, rootless containers, the credential, the monitoring account):
  docs/MARIADB.md.
#
# MySQL in podman containers: run  sudo podman ps -a  on a host, note the container NAME(S), then
  delete
# the "# " in front of the lines you need and replace CHANGE-ME. Attach the "MariaDB monitor"
  credential
# (the read-only DB account) to the Health check, Database health and Troubleshoot job templates.
# check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME                  # one container ...
# check_mariadb_containers: [CHANGE-ME, CHANGE-ME-2]  # ... or several on the same host
# check_mariadb_container_discover: true              # ... or find every mysql/mariadb/percona
  container
# check_mariadb_container_user: CHANGE-ME             # rootless: the account that owns the
  containers
# check_mariadb_group_size: 3                         # Group Replication: members expected ONLINE
```

netapp.yml (AAP group netapp)

YAML

```
# YOUR SETTINGS FOR THE NETAPP HOSTS (AAP group netapp).
# Nothing is needed yet: all.yml already makes these hosts diagnose-only for ACT and skipped by
# patching (vendor-managed). They are checked like any other RHEL host.
```

rhel8_all.yml (AAP group rhel8_all)

YAML

```
# YOUR SETTINGS FOR THE RHEL 8 HOSTS (AAP group rhel8_all).
# Only needed if AAP's Ansible is 2.17 or newer: it cannot use RHEL 8's built-in Python 3.6.
  Install
# python39 on these hosts (from Satellite), then delete the "# " below. ACT also needs Python
  3.8+.
# ansible_python_interpreter: /usr/bin/python3.9
```

sat.yml (AAP group sat)

YAML

```yaml
# YOUR SETTINGS FOR THE SATELLITE HOSTS (AAP group sat). ACT never fixes anything here, and
# patching skips these hosts: Satellite is upgraded with satellite-maintain (all.yml). The
  certificate
# report and the certificate renewal leave them out (cert_skip_groups): Satellite renews its own
# certificates with satellite-installer. The health check still checks the web UI's certificate
  below.
check_disk_overrides:
  /var: {warn: 80, crit: 90}             # Satellite keeps its content (Pulp) under /var
check_certs_endpoints:
  - {name: Satellite web UI, host: 127.0.0.1, port: 443}
# Services that must run. Check the names first: satellite-maintain service list
# check_services_required_extra: [foreman, httpd, postgresql, pulpcore-api, pulpcore-content]
```

stigman.yml (AAP group stigman)

YAML

```yaml
# YOUR SETTINGS FOR THE STIG MANAGER HOSTS (AAP group stigman).
# Service watch and STIG Manager deploy run on this group by themselves (no target needed).
# Service watch finds the containers that should be running by itself (root's and each user's).
# This list is watched in any case, first and in this order (dependencies first), and a container
# in it that is missing is reported. A container name (sudo podman ps -a) or a service name
# without .service: how each one is run (systemd unit or plain podman) is found by itself. Add
# `unit: NAME.service` under one only if the job output shows the wrong unit, and `user: NAME`
# for a container of a user (rootless podman).
# These are the containers STIG Manager deploy (roles/stigman_stack) makes; Keycloak is your own,
  elsewhere. A STIG
# Manager you set up yourself: its container names instead (sudo podman ps -a), e.g. a Keycloak
  container of its own.
watch_containers:
  - name: stigman-mysql
  - name: stigman-api
  - name: stigman-nginx
check_certs_endpoints:                   # the certificate browsers see
  - {name: STIG Manager, host: 127.0.0.1, port: 443}
# check_certs_files: [/etc/CHANGE-ME/*.crt]   # also check the certificate files, if you like
```